



B.M.S. COLLEGE OF ENGINEERING Bengaluru-560019.

Autonomous College, affiliated to
Visvesvaraya Technological University, Belgaum



PROJECT WORK-2 REPORT ON

Hardware-Accelerated One Round Cipher for Secure Image Transfer

Submitted in partial fulfilment of the requirement for completion of
PROJECT WORK-2 [23EC7PWPJ2]

Submitted by

Chiranjivi N Patil	1BM22EC071
Darshan Ghali	1BM22EC074
Kedar Desai	1BM22EC109
Anirudha H Kulkarni	1BM22EC302

Under the guidance of
Mrs. Pooja A P
Assistant Professor, Dept. of ECE
BMS College of Engineering, Bengaluru

Academic Year
2025 – 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bangalore-560019

Autonomous College, affiliated to Visvesvaraya Technological University, Belgaum

BMS COLLEGE OF ENGINEERING

Autonomous college, affiliated to VTU

Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certified that the Project Work - 2 entitled “**Hardware-Accelerated One Round Cipher for Secure Image Transfer**” is a Bonafide work carried out by **Chiranjivi N Patil (1BM22EC071), Darshan Ghali (1BM22EC074), Kedar Desai (1BM22EC109) and Anirudha H Kulkarni (1BM22EC302)**, submitted in fulfilment of the requirement for completion of the PROJECT WORK 2 [23EC7PWPJ2] of Bachelor of Engineering in Electronics and Communication during the academic year 2025-26. The Project Work 2 report has been approved as it satisfies the academic requirements.

Guide

(Mrs. Pooja A. P.)

Assistant Professor

Department of ECE

BMS College of Engineering

Head of Department

(Dr. K. P. Lakshmi)

Professor and Head

Department of ECE

BMS College of Engineering

(Dr. Bheemsha Arya)

Principal

BMS College of Engineering

External Viva

Name of the Examiners

Signature with Date

1.

2.

DECLARATION

We, **Chiranjivi N Patil (1BM22EC071)**, **Darshan Ghali (1BM22EC074)**, **Kedar Desai (1BM22EC109)** and **Anirudha H Kulkarni (1BM22EC302)**, hereby declare that the Project Work 2 entitled “**Hardware-Accelerated One Round Cipher for Secure Image Transfer**” is a Bonafide work and has been carried out by us under the guidance of **Mrs. Pooja A P**, Assistant Professor, Department of Electronics and Communication Engineering, BMS College of Engineering, Bengaluru submitted in fulfilment of the requirement for completion of PROJECT WORK 2 [23EC7PWPJ2] of Bachelor of Engineering in Electronics and Communication during the academic year 2025-26. The Project Work 2 report has been approved as it satisfies the academic requirements in Electronics and Communication engineering, Visvesvaraya Technological University, Belagavi, during the academic year 2025-26.

We further declare that, to the best of our knowledge and belief, this Project Work 2 has not been submitted either in part or in full to any other university.

Place: Bengaluru

Date:

Chiranjivi N Patil : 1BM22EC071

Darshan Ghali : 1BM22EC074

Kedar Desai : 1BM22EC109

Anirudha H Kulkarni : 1BM22EC302

ACKNOWLEDGEMENTS

We take this opportunity to express our profound gratitude to the respected principal, Dr. Bheemsha Arya, BMS College of Engineering for providing a congenial environment to work in. Our sincere gratitude to Dr. K.P. Lakshmi, Head of the Department, Electronics and Communication Engineering for encouraging and providing opportunity to carry Mini Project in the department. We heartfully thank our guide, Mrs. Pooja A.P., for her guidance and constant encouragement throughout the course of this Project Work 2 without which this project would not be successful. A number of personalities, in their own capacities have helped us in carrying out this Project Work 2. We would like to take this opportunity to thank them all. Last but not the least, we thank our friends and family for their encouragement and help in accomplishing the objective of the Project Work 2.

- 1) Chiranjivi N Patil : 1BM22EC071
- 2) Darshan Ghali : 1BM22EC074
- 3) Kedar Desai : 1BM22EC109
- 4) Anirudha H Kulkarni : 1BM22EC302

ABSTRACT

The rapid growth of multimedia communication systems has increased the demand for secure and real-time transmission of digital images, particularly in applications such as surveillance and medical imaging, where low latency and hardware efficiency are critical. Conventional multi-round cryptographic algorithms often introduce high computational overhead, making them unsuitable for such environments. This project presents a hardware-accelerated one-round cipher for secure RGB image transfer, designed to achieve an efficient balance between security, speed, and computational complexity. The proposed approach employs a single-round confusion–diffusion-based encryption scheme in which RGB images are decomposed into individual colour channels and processed using pixel-level permutation and value modification to reduce spatial redundancy and obscure statistical patterns. To support real-time operation, the encryption and decryption processes are implemented using GPU-based parallel processing, enabling fast handling of high-resolution images. The encrypted image is transmitted wirelessly using ESP32 modules, and the original image is reconstructed at the receiver by reversing the encryption process with the same dynamically generated key. Experimental results demonstrate low latency, high throughput, and effective security characteristics, validated using entropy, avalanche effect, and execution time analysis, confirming the suitability of the proposed one-round cipher for secure image transmission on hardware-accelerated platforms.

Keywords: *Image Encryption, One-Round Cipher, Hardware Acceleration, Confusion–Diffusion, RGB Image Processing, GPU-Based Parallel Processing, Real-Time Image Transmission, ESP32 Wireless Communication, Low Latency, Avalanche Effect*

TABLE OF CONTENTS

Abstract	i
List of Figures	iv
List of Tables	v
Chapter 1. Introduction	1
Chapter 2. Literature Survey.....	3
Chapter 3. Problem Analysis & Solution.....	6
3.1 Problem Definition.....	6
3.2 Proposed Solution	7
3.2.1 Image Encryption Algorithm	7
3.2.2 GPU-Based Acceleration of Encryption and Decryption.....	8
3.2.3 Secure Image Transmission Using ESP32 Wi-Fi Modules.....	8
Chapter 4. Methodology & Implementation	9
4.1 The Encryption and Decryption Algorithm	9
4.1.1 The S-Box.....	12
4.2 Overview of GPU Architecture and its Parallel Processing Capability	13
4.2.1 CPU–GPU Communication and Heterogeneous Memory Systems.....	14
4.2.2 Typical CUDA Program Flow	15
4.2.3 CUDA Kernels and Parallel Execution Model	16
4.2.4 Thread Organization and Execution Hierarchy	17
4.2.5 Warp-Level Execution and SIMT Model.....	18
4.2.6 GPU Computation Hierarchy	18
4.2.7 CUDA Memory Model Overview.....	19
4.3 Transfer of Files using ESP32.....	20
4.4 Implementation.....	21
4.4.1 Vivado Block Diagram.....	21
4.4.2 FPGA Implementation.....	23
4.4.3 Real-Time Implementation.....	24

Chapter 5. Results & Discussion.....	25
5.1 Results of Real-Time Image Encryption and Decryption	25
5.2 Vivado Simulation Waveform	26
5.3 Avalanche Calculation	27
5.4 Entropy Test	28
5.5 Real-Time GPU Results	28
5.6 Results from ESP-32	30
 Chapter 6. Conclusion & Future Trends.....	 32
6.1 Conclusion.....	32
6.2 Future Trends	33

References

Plagiarism Report

LIST OF FIGURES

Figure No.	Description of Figure	Page No.
Figure 4.1	Proposed One-Round Encryption Algorithm	12
Figure 4.2	Proposed One-Round Decryption Algorithm	12
Figure 4.3	CPU-GPU Data Communication	14
Figure 4.4	Typical CUDA Program Flow	16
Figure 4.5	Thread Organization	17
Figure 4.6	GPU Memory Allocation	19
Figure 4.7	Encryption Block Diagram (done on Vivado)	21
Figure 4.8	Decryption Block Diagram (done on Vivado)	22
Figure 4.9	FPGA Implementation of the Algorithm	23
Figure 4.10	Real-Time Secure Image Transmission	24
Figure 5.1	Algorithm Simulation on Vivado Software	26
Figure 5.2	Calculation of Avalanche for the Algorithm	27
Figure 5.3	Entropy Test Results	28
Figure 5.4	Encryption on GPU	28
Figure 5.5	Decryption on GPU	29
Figure 5.6	File Transfer Website	30
Figure 5.7	ESP32 Statistics 1	30
Figure 5.8	ESP32 Statistics 2	30

LIST OF TABLES

Table No.	Description of Table	Page No.
Table 4.1	A 4-bit to 4-bit Substitution Box	13
Table 5.1	Table of Processed Images	25

Chapter 1:

INTRODUCTION

The exponential growth in digital imaging and multimedia communication has significantly transformed the way visual information is generated, processed, and transmitted across modern communication systems. Images today form a critical component of applications such as surveillance systems, satellite imaging, medical diagnostics, autonomous vehicles, and wireless multimedia networks. With advancements in sensor technology and communication infrastructure, these systems increasingly rely on high-resolution and high-frame-rate image data to enable accurate analysis, decision-making, and real-time monitoring.

As the volume and sensitivity of image data continue to increase, ensuring confidentiality, integrity, and authenticity during transmission has become a fundamental requirement. Images transmitted over networks often contain sensitive or mission-critical information, making them valuable targets for adversaries. Unauthorized access or manipulation of such data can compromise privacy, affect system reliability, or lead to severe operational consequences, especially in domains such as defence, healthcare, and critical infrastructure monitoring.

Alongside technological advancements, the threat landscape surrounding multimedia communication has also expanded. Image data transmitted over public or wireless channels is increasingly exposed to security risks such as eavesdropping, tampering, man-in-the-middle attacks, and unauthorized interception. Wireless transmission, in particular, introduces additional vulnerabilities due to its broadcast nature, making robust encryption mechanisms essential for protecting visual information against malicious entities.

Conventional encryption algorithms such as the Advanced Encryption Standard (AES) and Data Encryption Standard (DES) are widely adopted due to their strong security foundations and extensive cryptographic analysis. These algorithms, however, were primarily designed for general-purpose data encryption and do not specifically account for the structural characteristics of image data. Images exhibit high spatial redundancy and predictable statistical distributions, which can limit the efficiency of traditional cryptographic techniques when directly applied. Furthermore, standard block ciphers typically employ multiple encryption rounds involving complex substitutions, permutations, and key scheduling operations. While

these multi-round architectures enhance security, they also introduce significant computational overhead. In the context of real-time image processing, such overhead manifests as increased latency, higher memory consumption, and elevated power usage, making these algorithms less suitable for time-sensitive or resource-constrained environments.

Software-based encryption implementations running on general-purpose processors further exacerbate these limitations. The sequential execution model of conventional software, combined with limited instruction-level parallelism and shared processing resources, can restrict throughput when handling large image datasets. For applications that demand low-latency transmission, such as live video streaming, medical imaging systems, or remote sensing platforms, this performance bottlenecks become increasingly pronounced.

Another critical consideration arises when encryption is deployed in hardware-constrained environments. Embedded and portable systems often operate under strict constraints on logic resources, operating frequency, and power consumption. Integrating strong encryption mechanisms within such platforms therefore requires careful architectural choices to avoid excessive utilization of hardware resources or degradation of system performance.

As image resolutions and data rates continue to scale upward, the limitations of conventional encryption schemes become more evident. High-definition and ultra-high-definition imaging systems generate substantial data volumes that must be securely processed and transmitted within tight timing constraints. This trend highlights the growing importance of lightweight encryption techniques that can adapt to increasing data demands without imposing excessive computational or energy costs.

From a broader perspective, existing cryptographic approaches reveal a notable gap between the requirements of high-speed image transfer and the capabilities of traditional encryption systems. Achieving strong security while maintaining real-time performance and hardware efficiency remains a challenging objective, particularly in systems where encryption must coexist with other compute-intensive tasks. This evolving landscape underscores the need for exploring image-specific encryption strategies that align with both security and performance requirements. Approaches that reduce computational complexity, exploit parallelism, and support efficient hardware implementation are increasingly relevant as multimedia communication systems continue to advance in scale, speed, and functional complexity.

Chapter 2:

LITERATURE SURVEY

Below are some of the literatures and IEEE research papers which we went through before proposing the solution:

- [1] Noura, Hassan & Chehab, Ali & Sleem, Lama & Noura, Mohamad & Couturier, Raphaël & Mansour, Mohammad. (2018). One Round Cipher Algorithm for Multimedia IoT Devices. Multimedia Tools and Applications. 77. 10.1007/s11042-018-5660-y.**

With the rapid expansion of IoT devices, security and privacy have become major concerns, especially in data-sensitive and multimedia applications. Traditional cryptographic algorithms are often too heavy for resource-constrained devices, as they require multiple computation rounds and consume significant processing time and energy. These limitations are more severe with multimedia data, which demands real-time performance. To address this, a lightweight, single-round cipher algorithm was developed using dynamic key generation, substitution tables, permutation, and pseudo-random matrices. The approach offers strong security, low error propagation, and high speed, making it well-suited for multimedia encryption on constrained hardware.

- [2] Bogdanov, A. et al. (2007). PRESENT: An Ultra-Lightweight Block Cipher. In: Paillier, P., Verbauwhede, I. (eds) Cryptographic Hardware and Embedded Systems - CHES 2007. CHES 2007. Lecture Notes in Computer Science, vol 4727. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-74735-2_31**

With the establishment of the AES the need for new block ciphers has been greatly diminished; for almost all block cipher applications the AES is an excellent and preferred choice. However, despite recent implementation advances, the AES is not suitable for extremely constrained environments such as RFID tags and sensor networks. In this paper we describe an ultra-lightweight block cipher, present. Both security and hardware efficiency have been equally important during the design of the cipher and at 1570 GE, the hardware requirements for present are competitive with today's leading compact stream ciphers.

- [3] N. Wan and Y. Zhang, "Digital Image Encryption and Decryption Algorithm Based on Optimization and Fusion Strategy," *2020 13th International Symposium on Computational Intelligence and Design (ISCID)*, Hangzhou, China, 2020, pp. 244-248, doi: 10.1109/ISCID51228.2020.00061.

Chaotic systems have attracted significant attention in digital image encryption due to their large key space and strong resistance to plaintext attacks. However, existing methods face two major challenges: insufficient chaos caused by poor initial parameter selection, and the vulnerability of using chaotic encryption alone. This paper proposes an optimized image encryption and decryption algorithm based on a fusion strategy. Genetic algorithms are used to optimize the parameters of the chaotic Logistic map, enhancing sequence unpredictability. The encryption process involves two-level permutation and one-level diffusion, followed by a matching decryption method. Performance evaluations confirm the algorithm's security and effectiveness.

- [4] G. R, P. G, S. B, R. Krishnaprasanna, V. G and V. S. Pandi, "A Novel Design of an Image Encryption and Decryption Scheme Using Enhanced Cybersecurity Principles," *2023 International Conference on Emerging Research in Computational Science (ICERCS)*, Coimbatore, India, 2023, pp. 1-6, doi: 10.1109/ICERCS57948.2023.10434166.

With the rapid expansion of digital communication, securing sensitive image data has become critically important. As images are increasingly used for storing and sharing confidential information, encryption serves as a vital tool to prevent unauthorized access and protect privacy. This study proposes an improved image encryption method based on a modified Enhanced Image Cryptography Principle (EICP), incorporating a key stream generator to enhance security and performance. The approach is designed for secure transmission over vulnerable networks and is cross-validated against the RSA algorithm to evaluate its effectiveness. Results show improved encryption reliability and efficiency compared to traditional methods.

- [5] Gabr, M., Alexan, W., & Moussa, K. (2022). Image Encryption Through CA, Chaos and Lucas Sequence Based S-Box. 2022 IEEE International Conference on Microelectronics (ICM). <https://doi.org/10.1109/ICM56065.2022.10005469>**

The increasing demand for effective image encryption has driven research toward chaotic functions and mathematical sequences to enhance security. This thesis proposes a three-stage image encryption scheme combining the Lucas sequence, an S-Box, and a Tan Variant Logistic map to fulfill Shannon's confusion and diffusion criteria. Performance evaluations show the scheme offers strong resistance to various attacks with low computational cost, achieving high entropy, low correlation coefficients, and a large key space ($\sim 2^{266}$). Encryption and decryption speeds average 0.43 and 0.49 Mbps, respectively, demonstrating comparable or superior results to existing methods.

- [6] Nair, Bharath V et al. "Deep Learning and Chaos: A combined Approach To Image Encryption and Decryption." *ArXiv* abs/2406.16792 (2024): n. pag.**

This paper presents a novel image encryption and decryption algorithm that combines a 3D hyperchaotic map, 2D memristor map, and Convolutional Neural Network (CNN) to achieve robust security and high efficiency. The method scrambles grayscale images using hyperchaotic sequences and enhances security through CNN-based pattern learning. Key sensitivity analysis and extensive statistical tests confirm the algorithm's strong resistance to unauthorized decryption and noise, including Gaussian disturbances. Compared to traditional single-image encryption methods, this hybrid approach improves encryption robustness, visual security, and computational performance, making it well-suited for secure image transmission and storage in big data environments.

Chapter 3:

PROBLEM ANALYSIS & SOLUTION

3.1 Problem Definition:

The extensive adoption of digital images in critical domains such as surveillance, medical imaging, remote sensing, and secure communication systems has amplified the importance of reliable image encryption mechanisms for protecting sensitive visual information. Unlike conventional text-based data, digital images are characterized by large data sizes, high spatial redundancy, and strong statistical correlations among pixels, which pose significant challenges to traditional encryption approaches when applied directly. Although well-established cryptographic algorithms such as AES and DES provide strong theoretical security, they are primarily designed for general-purpose data encryption and rely on multiple computational rounds, complex key scheduling, and intensive processing operations. These characteristics introduce substantial latency, high energy consumption, and increased hardware resource utilization, making them unsuitable for real-time image encryption in environments with strict performance, power, and resource constraints, such as embedded systems and hardware-accelerated platforms. In recent years, alternative encryption techniques based on chaos theory and hybrid models have been explored to address some of these limitations; however, such approaches often suffer from insufficient key sensitivity, vulnerability to certain cryptanalytic attacks, degradation in decryption quality under noisy conditions, and increased computational or hardware complexity when incorporating advanced chaotic functions or learning-based components. Furthermore, practical image transmission applications demand encryption schemes that are scalable across varying image resolutions and formats, adaptable to different hardware architectures, and capable of balancing security strength with processing efficiency. Achieving this balance remains a significant challenge, as overly complex encryption algorithms risk failing real-time deployment requirements, while lightweight schemes may compromise security robustness. Consequently, there exists a critical need for an image encryption and decryption solution that effectively addresses the unique structural properties of image data, maintains strong security characteristics through high key sensitivity and effective confusion–diffusion mechanisms, minimizes computational and energy overhead,

and remains feasible for implementation on resource-constrained and hardware-accelerated platforms to enable secure and efficient image transfer in modern multimedia systems. Additionally, the deterministic structure and predictable statistical behaviour of image data demand encryption approaches that specifically address pixel-level correlations rather than treating images as generic binary streams. In scenarios involving continuous or bulk image transmission, even minor inefficiencies in encryption can accumulate into significant performance degradation. The lack of uniform standards for image-oriented encryption further complicates the selection of suitable techniques for real-world deployment. Interoperability with diverse hardware platforms and communication environments also remains an unresolved concern in existing approaches. These limitations collectively highlight the complexity of designing an image encryption framework that aligns security requirements with practical system constraints.

3.2 Proposed Solution:

3.2.1 Image Encryption Algorithm:

The proposed solution focuses on the secure encryption of RGB images intended for transmission between two servers. The input image is initially decomposed into its three primary colour components, Red, Green, and Blue allowing independent and structured processing of image data. This channel-wise separation facilitates efficient manipulation of pixel information and supports parallel execution of encryption operations.

The encryption algorithm is built around two fundamental cryptographic stages: confusion and diffusion. In the confusion stage, pixel positions within each RGB channel are permuted, effectively scrambling the spatial arrangement of image data and obscuring the original image structure. This process significantly reduces spatial correlation among adjacent pixels, which is a common characteristic of unencrypted images.

Following confusion, the diffusion stage modifies pixel intensity values using simple mathematical operations such as XOR-based transformations. This step ensures that changes in pixel values are spread across the encrypted image, making it difficult to derive information from statistical properties. Together, the confusion and diffusion stages provide effective image scrambling while maintaining low computational complexity.

3.2.2 GPU-Based Acceleration of Encryption and Decryption:

To support real-time image processing requirements, the encryption and decryption operations are implemented using GPU acceleration. Image data naturally exhibits pixel-level parallelism, making it well suited for execution on graphics processing units. By offloading confusion and diffusion operations to the GPU, multiple pixels can be processed simultaneously, significantly improving execution efficiency compared to sequential software implementations.

The GPU implementation parallelizes encryption and decryption tasks across multiple processing cores, enabling faster handling of high-resolution RGB images. This parallel execution model helps reduce processing latency and ensures consistent performance during both encryption and decryption stages.

By leveraging GPU-based parallel processing, the proposed system efficiently manages computationally intensive image operations without becoming a performance bottleneck. This approach allows the image encryption framework to meet the timing requirements of real-time multimedia applications while maintaining computational efficiency.

3.2.3 Secure Image Transmission Using ESP32 Wi-Fi Modules:

Once the encryption process is completed, the encrypted image data is transmitted wirelessly from the sender to the receiver using ESP32 modules. The ESP32 provides a low-power and flexible wireless communication interface, enabling data transmission over Wi-Fi. In the proposed setup, the encrypted image data is segmented into packets and transmitted across the wireless channel.

The use of encrypted image transmission ensures that the image content remains protected during communication, even if the transmitted data is intercepted. The ESP32 modules serve solely as communication devices, handling wireless data transfer without altering or interpreting the encrypted content.

At the receiver end, the encrypted image data is collected and reassembled before undergoing the decryption process. By reversing the same confusion and diffusion operations, the original RGB image is reconstructed. This integration of image encryption, GPU-accelerated processing, and ESP32-based wireless communication forms a complete and practical system for secure image transfer in real-time and resource-constrained environments.

Chapter 4:

METHODOLOGY & IMPLEMENTATION

4.1 The Encryption and Decryption Algorithm:

The cipher proposed in this work is inspired by the Feistel structure, which is a widely adopted architecture in symmetric key cryptography and forms the foundation of many well-established block ciphers. A Feistel structure enables efficient encryption and decryption using the same algorithmic components, thereby providing simplicity of implementation while maintaining cryptographic strength. The designed cipher operates as a block cipher, where the plaintext is processed in fixed-size blocks rather than as a continuous stream of data.

Cryptographic ciphers, in general, can be classified into stream ciphers and block ciphers based on the manner in which plaintext is encrypted. Stream ciphers operate on plaintext bits individually in a sequential manner, whereas block ciphers divide the plaintext into blocks of equal length and encrypt each block independently. In block ciphers, security relies heavily on structured transformations that ensure both confusion and diffusion, as introduced by Claude Shannon. These two properties obscure the relationship between the plaintext, ciphertext, and key, thereby preventing statistical and structural attacks. The proposed cipher is designed with these principles as its core foundation.

In the proposed implementation, the input plaintext block is first divided into two equal halves, referred to as the left half (P1) and the right half (P2). Similarly, the secret encryption key is also partitioned into two equal components, namely K1 and K2, which are used in the encryption process. This symmetric partitioning facilitates balanced processing of data and allows the cipher to follow a Feistel-like round structure.

Each half of the plaintext undergoes a sequence of cryptographic operations that include exclusive-OR (XOR) operations, nonlinear substitution using an S-box, and XOR mixing with initialization vectors (IVs). XOR operations are computationally efficient and are commonly used in symmetric cryptographic algorithms due to their reversibility, which ensures that decryption can be performed without additional complexity.

To introduce nonlinearity and strengthen the cryptographic properties of the cipher, a Substitution Box (S-box) is incorporated into the design. The S-box is a fundamental building block in many symmetric encryption algorithms and plays a crucial role in ensuring both confusion and diffusion. It operates as a substitution table, wherein each input byte is mapped to a corresponding output byte according to a predefined fixed table.

By performing nonlinear substitutions, the S-box removes linear relationships between the plaintext and ciphertext, making cryptanalysis significantly more difficult. Even a small change in the input data can result in a substantial change in the output after passing through the S-box, thereby contributing to the avalanche effect. In the proposed cipher, the S-box is applied after initial XOR operations with the key, strengthening resistance to linear and differential attacks.

Following the XOR operation with the key and substitution through the S-box, the resulting intermediate values are further XORed with Initialization Vectors (IV1 and IV2) for the left and right halves, respectively. In cryptography, an initialization vector is an additional input used to introduce randomness into the encryption process. Its primary purpose is to ensure that identical plaintexts encrypted using the same key produce different ciphertexts.

The IV is typically required to be random or pseudorandom, and in some cases unpredictable or unique. This randomness is essential to achieve semantic security, a property that prevents attackers from deducing any meaningful patterns or relationships between encrypted messages. By incorporating initialization vectors into both halves of the plaintext, the proposed cipher ensures enhanced resistance against replay attacks and pattern-based cryptanalysis.

After completing the sequence of XOR operations, S-box substitutions, and IV mixing, the partially encrypted outputs obtained from the left and right halves are denoted as C1 and C2, respectively. As per the Feistel structure, these two halves are then swapped and concatenated to form the final encrypted block.

This swapping operation is key to the Feistel network, as it enables the same algorithm to be used for both encryption and decryption. The encryption output obtained after concatenation serves as the ciphertext and also acts as the input to the decryption process.

The decryption mechanism follows the reverse sequence of operations applied during encryption. Due to the symmetric nature of the cipher and the reversibility of XOR operations, decryption can be achieved without introducing any additional computational overhead. The same secret key is used for both encryption and decryption, making the cipher a symmetric-key encryption algorithm. This design ensures simplicity of hardware implementation while preserving functional correctness.

To extend the proposed cipher for secure image transmission, the encryption process is applied to RGB images, which are commonly used in digital imaging systems. In an RGB image, each pixel consists of three colour components, Red, Green, and Blue, each represented using 8 bits, leading to a total of 24 bits per pixel.

For instance, in a 256×256 image, each pixel comprises these 24 bits, which are separated into three individual 8-bit components corresponding to the R, G, and B channels. The proposed cipher is designed to accept 8-bit plaintext inputs and uses an 8-bit key, making it suitable for directly encrypting each colour component independently.

Before encryption, the image memory file is split into three distinct streams, each representing one of the RGB colour components. Each 8-bit data value is encrypted independently using the cipher. Once encrypted, the three processed components are recombined to form the encrypted image file. This approach ensures uniform encryption across all colour channels while preserving image dimensions and format.

The decryption process for the image follows the same splitting and combining methodology. The encrypted image file is divided into its respective RGB components, and each 8-bit encrypted value is passed through the decryption algorithm using the same secret key. The decrypted components are then recombined to reconstruct the original image. This methodology guarantees lossless recovery of image data.

The proposed cipher was initially implemented and verified using Verilog HDL. An Input Memory Module was designed to store the image memory file, which was subsequently fed into the splitting module responsible for separating the RGB components. The encrypted output was generated through the cipher module, and the decryption process was applied thereafter. The complete design was validated using block-level simulations. The simulation results

confirmed that the decrypted image was identical to the original input image, thereby verifying the functional correctness and reliability of the proposed encryption–decryption scheme.

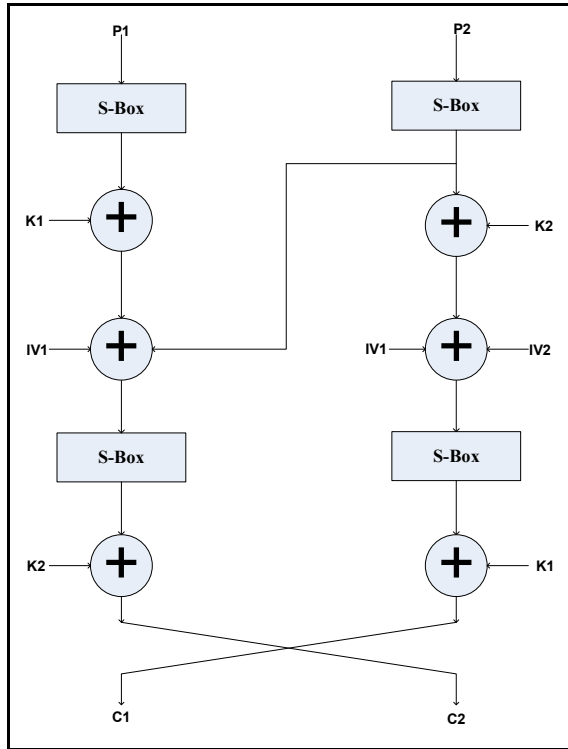


Fig 4.1. Proposed One-Round Encryption Algorithm

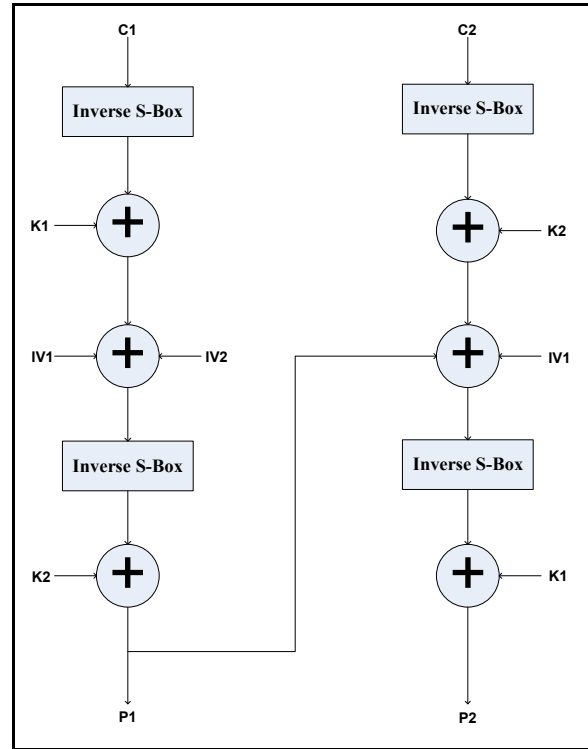


Fig 4.2. Proposed One-Round Decryption Algorithm

4.1.1 The S-Box:

A Substitution Box (S-box) is a fundamental component of symmetric key block ciphers and serves as the primary source of nonlinearity in the encryption process. It performs a substitution operation in which each fixed-size input value is replaced by a corresponding output value according to a predefined lookup table. In an 8-bit S-box, for instance, each of the 256 possible input values is uniquely mapped to another 8-bit output value. This nonlinear mapping is crucial for breaking linear relationships between the plaintext, ciphertext, and secret key. Without the use of an S-box, cryptographic algorithms relying solely on linear operations such as XOR would be vulnerable to linear and differential cryptanalysis.

The S-box significantly strengthens the confusion and diffusion properties of the cipher. By ensuring that a small change in the input or key results in a large and unpredictable change in

the output, the S-box contributes to the avalanche effect, enhancing overall security. In the proposed Feistel-based cipher, the S-box is applied after XOR operations with the secret key, ensuring that the substitution is key-dependent and resistant to pattern analysis. Additionally, S-boxes are efficient to implement in hardware, as they can be realized using lookup tables or combinational logic. Thus, the S-box plays a vital role in improving cryptographic strength without increasing computational overhead. A 4-bit S-box is utilized in our cipher that has mapping for 16 distinct input values.

The S-box used in the cipher is a 4-bit to 4-bit S-box $S: F_2^4 \rightarrow F_2^4$

The action of this box in hexadecimal notation is given by the following table:

Table 4.1. A 4-bit to 4-bit Substitution Box

x (Input)	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S(x)	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

4.2 Overview of GPU Architecture and its Parallel Processing Capability:

Graphics Processing Units (GPUs) are designed fundamentally differently from Central Processing Units (CPUs) to efficiently handle massive parallel workloads. While CPUs are optimized for low-latency execution of complex control-heavy tasks using a few powerful cores, GPUs focus on high-throughput execution by using thousands of simpler cores capable of executing the same operation on many data elements simultaneously. This architectural distinction makes GPUs highly suitable for data-parallel applications such as image processing, deep learning, scientific simulations, and cryptography. The Fig. 3, illustrates this separation clearly by showing the CPU with its local DRAM and the GPU with its own global memory, connected through the PCI Express (PCIe) bus. This separation forms the foundational concept of heterogeneous computing, where the CPU acts as the host orchestrating execution, and the GPU acts as the device performing parallel computations.

In the GPU architecture, the emphasis is on maximizing arithmetic intensity and throughput rather than minimizing execution latency for individual tasks. To achieve this, GPUs adopt the

Single Instruction, Multiple Threads (SIMT) execution model, where groups of threads execute the same instruction simultaneously on different data. Instead of complex control logic, branch prediction, and large caches, as commonly found in CPUs, GPUs devote a larger fraction of silicon area to arithmetic logic units (ALUs). This allows GPUs to schedule thousands of lightweight threads and hide memory latency through rapid context switching. The architectural model shown across the diagrams reinforces this philosophy by highlighting how work is offloaded in large chunks from the CPU and executed concurrently across many GPU threads.

4.2.1 CPU–GPU Communication and Heterogeneous Memory Systems

The Fig. 3, focuses on CPU–GPU communication, emphasizing the fact that the CPU and GPU generally have distinct physical memory spaces. The CPU accesses system DRAM, whereas the GPU accesses its own high-bandwidth global memory, typically implemented using GDDR or HBM technologies. Communication between these memory spaces occurs over the PCI Express bus, which, although fast, is significantly slower than on-chip memory accesses. This separation has critical performance implications and shapes how CUDA programs are written and optimized.

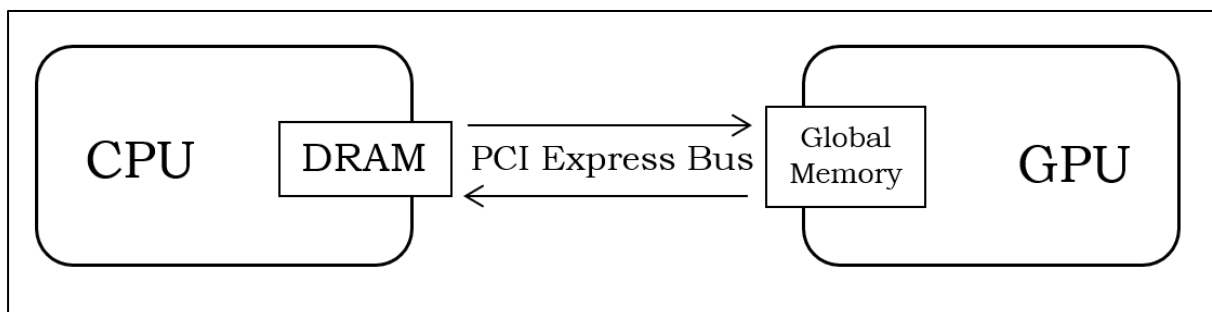


Fig 4.3. CPU-GPU Data Communication

Because the GPU cannot directly operate on CPU memory (in traditional discrete GPU systems), data must be explicitly transferred between the host (CPU) and device (GPU). This process introduces overhead and must be carefully managed to avoid performance bottlenecks. The diagram visually represents this by showing directional data movement arrows between CPU DRAM and GPU global memory. In practice, functions such as `cudaMemcpy()` are used to initiate these transfers. Minimizing unnecessary transfers and reusing data on the GPU as much as possible is a core optimization principle in CUDA programming.

The clear separation between CPU and GPU memory also enforces a programmer's awareness of data locality. Computation-heavy tasks with minimal data movement benefit most from GPU acceleration. Conversely, tasks with frequent small data transfers may perform better on the CPU due to PCIe overhead. Modern CUDA features such as Unified Memory abstract this separation to some extent, but the underlying architecture shown in the diagram remains conceptually important for understanding performance behaviour.

4.2.2 Typical CUDA Program Flow

The Fig. 4, depicts the typical execution flow of a CUDA-enabled application, capturing the lifecycle of data and computation across CPU and GPU domains. The program begins execution on the CPU, which is responsible for loading data from external sources such as a file system into host memory. This step highlights the CPU's role in handling I/O-bound and control-oriented tasks that are inefficient or unsupported on the GPU.

Once the data is available in CPU memory, it is explicitly transferred to GPU global memory. This transfer step marks the transition from serial, host-side execution to parallel, device-side computation. After the data is copied, the CPU launches a CUDA kernel, which is a function written to execute on the GPU. This kernel launch does not execute the code immediately on the CPU; instead, it instructs the GPU to schedule and execute thousands of threads in parallel using the SIMT model.

Following kernel execution, the computed results are stored in GPU global memory. These results are then copied back to CPU memory, completing the bidirectional communication cycle shown in the diagram. Finally, the CPU processes or displays the results, integrates them into the application pipeline, or writes them back to persistent storage. This five-step flow, load, copy to GPU, execute kernel, copy back, and use results, is fundamental to CUDA programming and provides a conceptual framework for understanding performance-critical paths within heterogeneous applications.

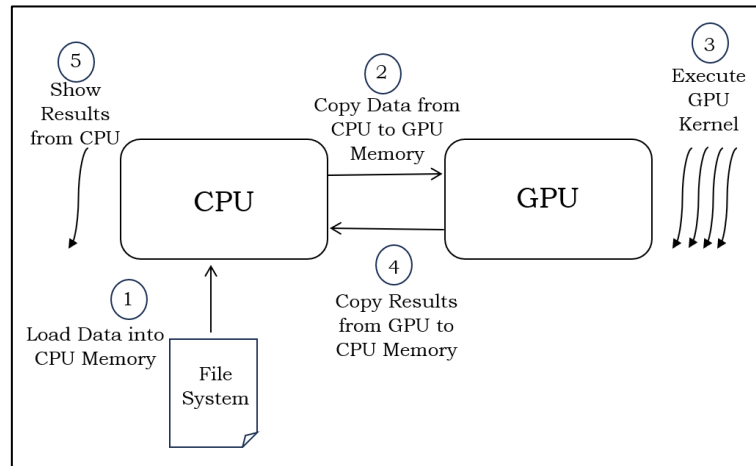


Fig 4.4. Typical CUDA Program Flow

4.2.3 CUDA Kernels and Parallel Execution Model

A CUDA kernel is the basic unit of execution on the GPU and represents a function that runs in parallel across many threads. Unlike traditional CPU functions that execute once per call, a CUDA kernel is invoked across a large number of threads simultaneously. Each thread executes the same kernel code but operates on different data elements based on its unique thread index. This approach enables fine-grained data parallelism, which is essential for achieving high throughput on GPUs.

The kernel execution model aligns directly with the Fig. 5, which illustrates how a kernel launched from the host is executed on the device as a grid of blocks. The CPU initiates the kernel launch, specifying the number of blocks and threads per block. The GPU runtime then distributes these blocks across available streaming multiprocessors for execution. From the programmer's perspective, this abstraction allows scalable parallel execution without explicit management of hardware resources such as individual cores.

CUDA kernels are designed to be lightweight and massively parallel. Since thousands of threads may be active simultaneously, each thread is expected to perform a relatively small amount of work. This contrasts with CPU threads, which are fewer in number and typically handle more complex workloads. The kernel-based execution model allows the GPU to interleave the execution of multiple warps, hiding memory latency by switching between active warps whenever one is stalled.

4.2.4 Thread Organization and Execution Hierarchy

The Fig. 5, introduces CUDA's hierarchical execution structure, which organizes parallelism into grids, blocks, warps, and threads. At the highest level, a kernel launch creates a grid, which is a collection of thread blocks. Each block contains multiple threads that can cooperate with each other through shared memory and synchronization primitives. Blocks are independent of one another, enabling scalable execution across GPUs with varying hardware resources.

Within each block, threads are organized into warps, which are groups of typically 32 threads that execute in lockstep. The warp is the fundamental execution unit in the GPU hardware. All threads within a warp execute the same instruction at the same time, although they may operate on different data. If threads within a warp diverge due to conditional branches, execution is serialized, leading to reduced efficiency. This behaviour underscores the importance of minimizing branch divergence in CUDA programs.

Threads are the smallest execution units and are identified by multi-dimensional indices, as shown in the zoomed-in portion of the diagram. These indices allow threads to map naturally to multidimensional data structures such as images and matrices. The hierarchical indexing scheme, combining grid, block, and thread indices, provides a flexible and intuitive way to express parallel algorithms while remaining hardware agnostic.

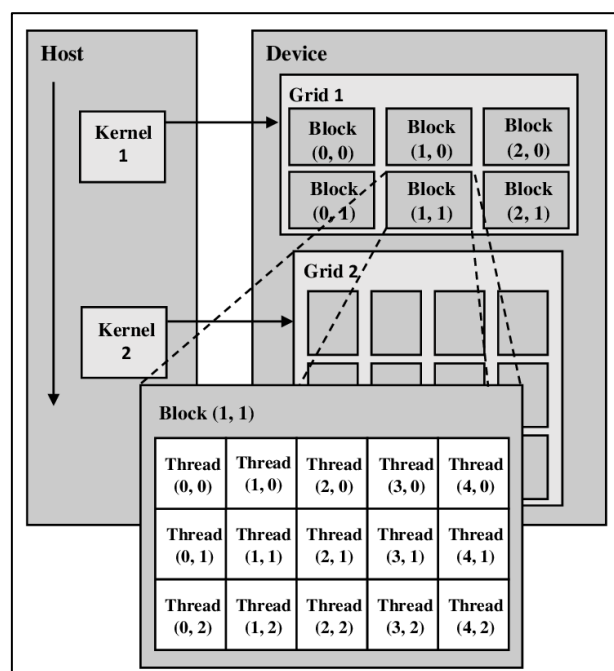


Fig 4.5. Thread Organization

4.2.5 Warp-Level Execution and SIMT Model

The concept of a warp is central to understanding GPU performance. Unlike CPU architectures that execute threads independently, GPUs execute threads in warps using the SIMT model. This means that a single instruction is issued to all threads in the warp simultaneously. The hardware ensures that each thread has its own registers and program counter, enabling conditional execution, but the physical execution remains synchronized at the instruction level.

Warp-level execution enables GPUs to achieve high efficiency by amortizing instruction fetch and decode overhead across multiple threads. However, it also imposes constraints on program structure. Control flow divergence within a warp leads to serialized execution of different code paths, reducing parallel efficiency. The diagram's depiction of threads grouped within blocks implicitly reflects this warp structure, even though warps themselves are not explicitly configured by the programmer.

Efficient CUDA programming requires awareness of warp behaviour, even when writing high-level code. Memory access patterns, control flow, and computation distribution should be designed such that threads within a warp follow similar execution paths and access nearby memory locations. This alignment allows the GPU to fully leverage its SIMD-like execution model.

4.2.6 GPU Computation Hierarchy

The GPU computation hierarchy consists of multiple levels, starting with the GPU device itself, which contains several streaming multiprocessors (SMs). Each SM is capable of executing multiple blocks concurrently, depending on resource availability such as registers and shared memory. This modular design allows GPUs to scale performance by increasing the number of SMs.

Within each SM, blocks are scheduled and executed by dividing their threads into warps. Each warp is issued instructions through warp schedulers, which manage execution and hide latency by switching between ready warps. Threads within warps execute on execution units that perform arithmetic and logical operations. This hierarchy, from GPU to SM to block to warp to thread, is reflected clearly in the third diagram and forms the backbone of CUDA's scalability and performance.

The independence of blocks enables portability across GPU architectures. A kernel written for a small GPU will also function correctly on a larger GPU, where more blocks can be executed concurrently. This abstraction allows programmers to focus on algorithmic parallelism rather than hardware-specific scheduling.

4.2.7 CUDA Memory Model Overview

The CUDA memory model provides multiple memory spaces, each with different performance characteristics and scopes. Global memory is the largest and most accessible memory space, but it has high latency. It is used for storing input data, intermediate results, and output data that must persist across kernel launches. The first diagram highlights global memory as the primary memory on the GPU, distinct from CPU DRAM.

Shared memory resides on-chip within each SM and is shared among threads in a block. It offers much lower latency than global memory and is ideal for data reuse and inter-thread communication. Proper use of shared memory can significantly improve performance by reducing redundant global memory accesses. Registers are the fastest memory space and are private to each thread, storing frequently used variables and temporary values.

Other memory spaces include constant memory and texture memory, which are optimized for specific access patterns. Constant memory is cached and efficient for read-only data shared across threads, while texture memory provides optimized hardware support for spatially local access patterns common in image processing. Understanding and exploiting the CUDA memory hierarchy is essential for writing high-performance GPU applications.

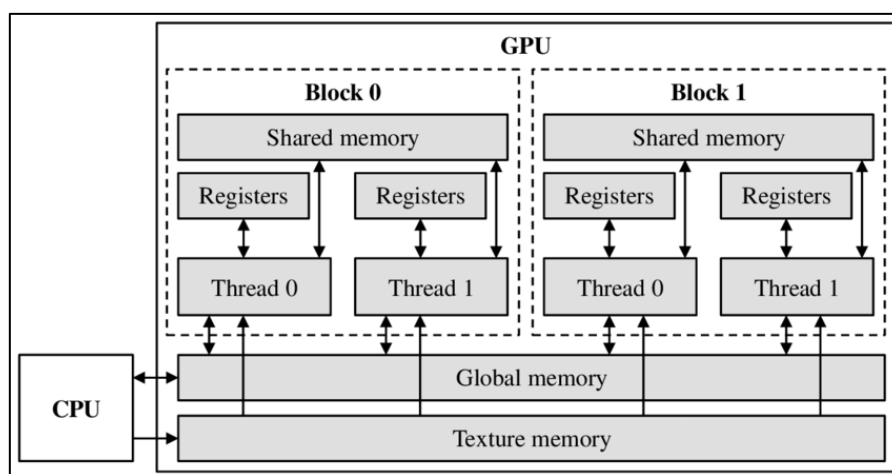


Fig 4.6. GPU Memory Allocation

4.3 Transfer of Files using ESP32:

The ESP32 enables file transfer by integrating networking, storage, and application services within a single embedded device. In this system, the ESP32 operates simultaneously as a wireless access point, a web server, and a storage unit, forming a self-contained micro-internet. This eliminates the need for external routers, servers, or cloud infrastructure, making the system highly suitable for standalone and embedded applications.

The workflow begins with the ESP32 configuring its Wi-Fi module in Access Point (AP) mode. In this mode, the ESP32 advertises itself as a wireless network and allows client devices such as laptops or smartphones to connect directly. Once a device connects, the ESP32 assigns it an IP address using an internal DHCP server. This establishes a local network in which the ESP32 acts as the central communication node, similar to a conventional router.

For file storage, the ESP32 utilizes its onboard flash memory through the SPI Flash File System (SPIFFS). A dedicated portion of the flash memory is allocated exclusively for file storage based on the selected partition scheme. SPIFFS abstracts the low-level flash operations and provides a simple file-based interface, handling wear levelling and logical-to-physical memory mapping automatically. This allows reliable storage and retrieval of files despite the limited write endurance of flash memory.

Once the network and storage layers are active, the application layer is implemented using an embedded web server running on the ESP32. The server continuously listens for incoming HTTP requests on a predefined port. These requests are serviced in an event-driven manner, allowing the ESP32 to respond to client actions such as opening a webpage, uploading a file, or downloading stored content without blocking normal operation.

During file upload, the client device sends an HTTP POST request using the multipart/form-data format. Instead of loading the entire file into RAM, the ESP32 processes the incoming data as a continuous stream. The file is divided into small packets by the network stack, and each packet is written directly to flash memory as soon as it arrives. After writing, the corresponding buffer space in RAM is freed, enabling the ESP32 to handle files much larger than its available memory. This streaming approach ensures efficient memory usage and stable operation even for multi-megabyte files.

At the completion of the upload process, the ESP32 detects the termination boundary sent by the client and closes the corresponding file in the flash system, ensuring data integrity. Optional timing calculations can be performed to estimate transfer speed based on the total file size and transmission duration.

For downloading files, the ESP32 follows a chunked streaming approach. When a client requests a file, the ESP32 reads small blocks of data from flash memory and transmits them sequentially over Wi-Fi. Each block is acknowledged by the client before the next block is sent. This controlled data flow prevents buffer overflows and ensures reliable transmission despite limited system resources.

Overall, the ESP32 file transfer workflow relies on efficient packet buffering, streaming-based data handling, and layered system operation. By combining wireless networking, file system management, and web-based communication into a single embedded device, the ESP32 provides a lightweight yet powerful platform for local file hosting and transfer in embedded and IoT applications.

4.4 Implementation:

4.4.1 Vivado Block Diagram:

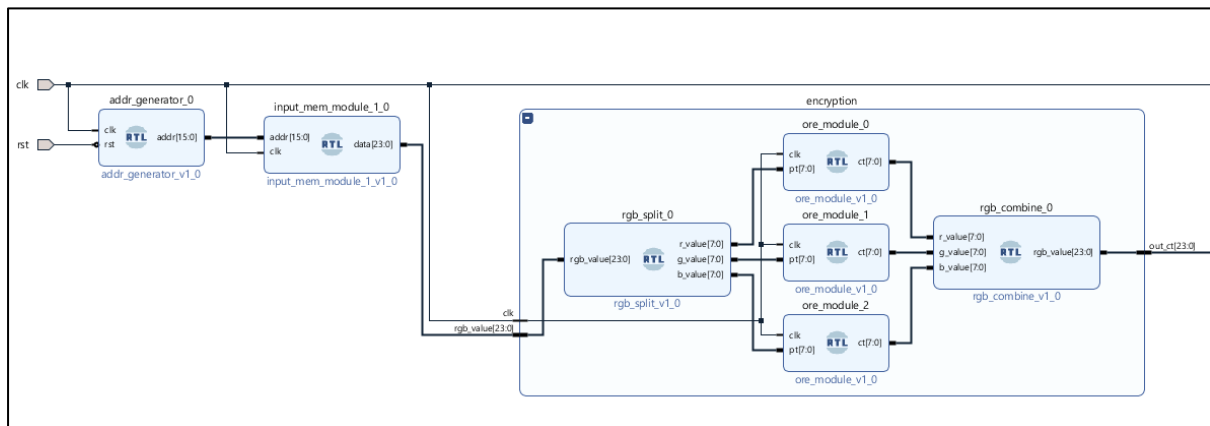


Fig 4.7. Encryption Block Diagram (done on Vivado)

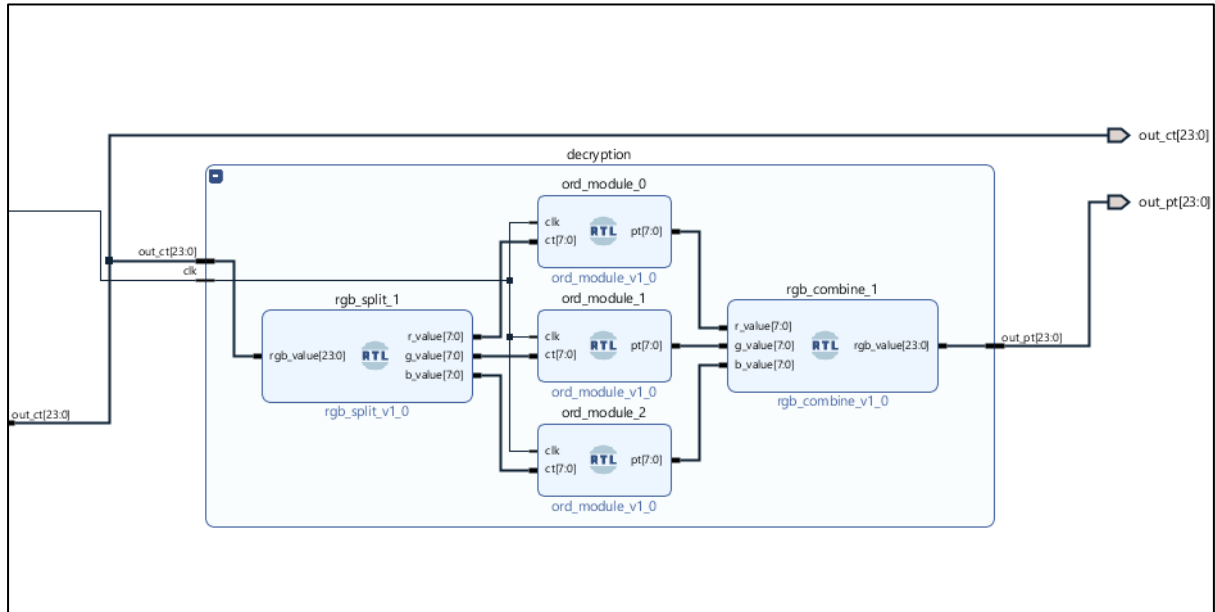


Fig 4.8. Decryption Block Diagram (done on Vivado)

In the encryption architecture, pixel data is read sequentially from memory using an address generator, which supplies addresses to the input memory module to fetch 24-bit RGB pixel values. Each pixel is then split into its red, green, and blue components using the RGB split module. These individual colour channels are processed in parallel by three identical encryption cores (ORE modules), each taking an 8-bit plaintext value and producing an encrypted ciphertext based on the encryption logic. Parallel processing ensures high throughput and efficient utilization of hardware resources. Once encrypted, the individual R, G, and B ciphertext components are recombined using an RGB combine module to form a 24-bit encrypted pixel, which is routed to the output. This modular and parallel structure enables fast, pixel-level encryption suitable for real-time image processing on hardware.

The decryption architecture mirrors the encryption flow to recover the original image data. The incoming 24-bit encrypted pixel stream is first split into its red, green, and blue ciphertext components using an RGB split module. Each colour channel is then passed independently through a corresponding decryption core (ORD module), which reverses the encryption operation to retrieve the original 8-bit plaintext values. These decryption operations are performed in parallel, maintaining the same throughput as the encryption stage. After decryption, the recovered R, G, and B components are recombined using an RGB combine module to reconstruct the original 24-bit pixel. This symmetric and pipelined design ensures accurate reconstruction of the image, preserving timing alignment with the encrypted data.

4.4.2 FPGA Implementation:

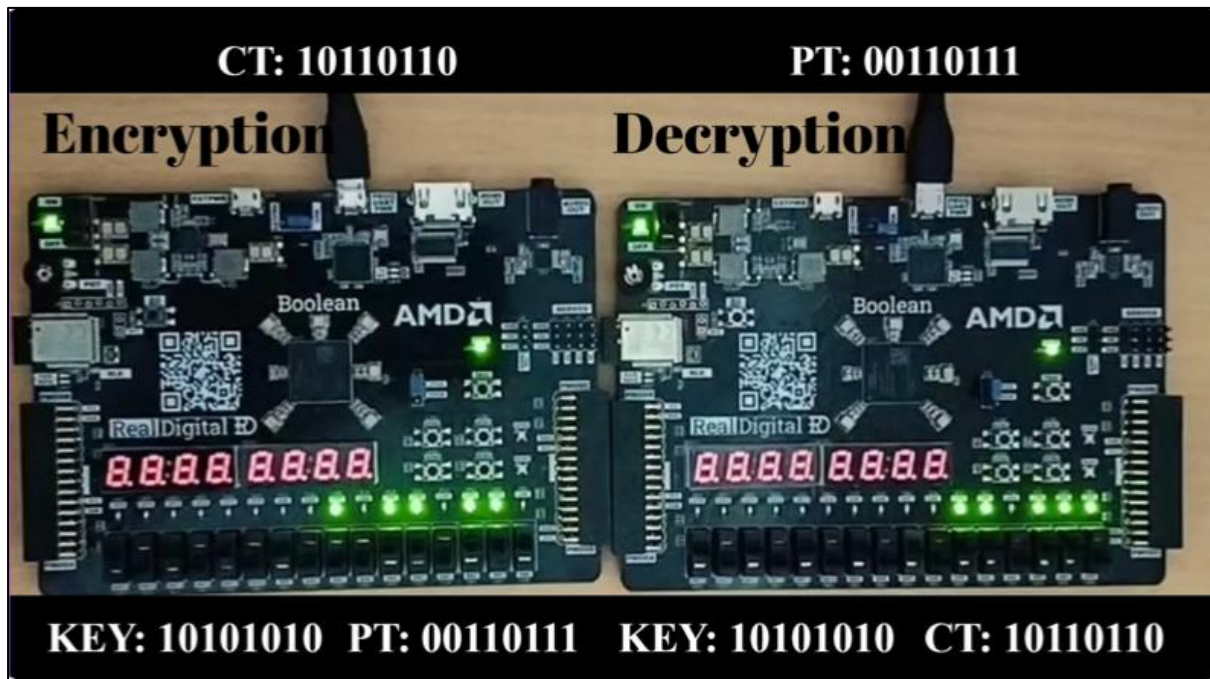


Fig 4.9. FPGA Implementation of the Algorithm

The Boolean FPGA boards were used as a dedicated hardware validation platform to thoroughly test and demonstrate the correctness of the proposed encryption and decryption algorithm. The complete logic was described in RTL and synthesized onto the Boolean boards, where user-defined inputs such as plaintext and key values were applied through onboard switches. The corresponding encrypted and decrypted outputs were monitored in real time using LEDs and seven-segment displays, enabling clear observation of bit-level transformations. This hardware-based testing allowed verification of functional accuracy, proper synchronization between encryption and decryption modules, and correct operation under actual clocked conditions, which cannot be fully captured through simulation alone. The use of Boolean boards was strictly limited to algorithm verification and proof-of-functionality, serving as a reliable bridge between simulation results and future system-level or image-based implementations. Additionally, testing on the Boolean boards helped identify timing inconsistencies, propagation delays, and integration issues at an early stage. It also ensured that the design met FPGA resource and timing constraints before scaling the algorithm for image-level processing. Overall, this approach strengthened confidence in the hardware feasibility and robustness of the proposed design.

4.4.3 Real-Time Implementation:

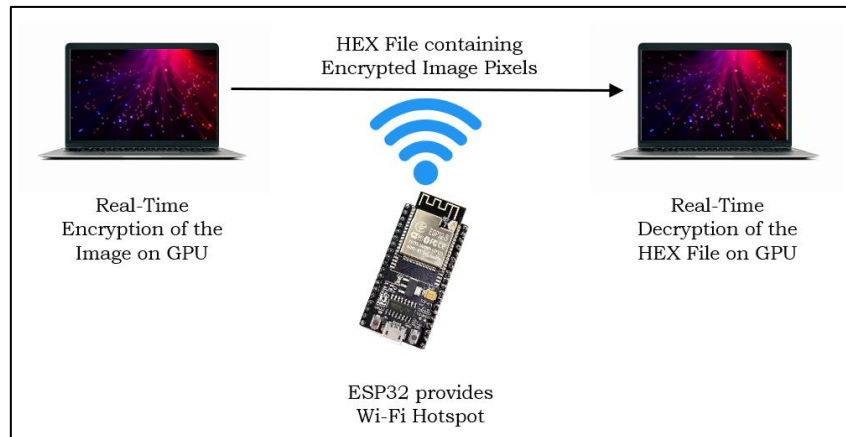


Fig 4.10. Real-Time Secure Image Transmission

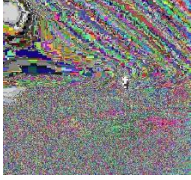
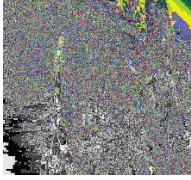
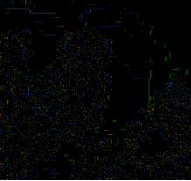
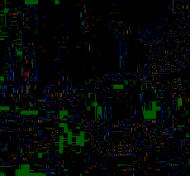
Real-time secure image transmission is accomplished by integrating GPU-based cryptographic processing with wireless communication provided by an ESP32 hotspot. In this workflow, the original image is encrypted in real time on the transmitter system using GPU parallelism, and the encrypted image pixels are converted into a HEX file format for efficient streaming. The ESP32 operates as a standalone Wi-Fi access point, enabling direct and secure transmission of the encrypted data without relying on external network infrastructure. At the receiver end, the transmitted HEX file is decoded and decrypted in real time on the GPU to reconstruct the original image with minimal latency. This architecture leverages the massive parallel processing capability of GPUs to meet real-time constraints while offloading communication control to a lightweight embedded device. The use of encrypted intermediate file formats ensures data confidentiality even during transmission. Overall, the system demonstrates a scalable and low-latency solution for secure image exchange in bandwidth- and security-critical environments.

Chapter 5:

RESULTS & DISCUSSION

5.1 Results of Real-Time Image Encryption and Decryption:

Table 5.1 Table of Processed Images

	Image 1	Image 2	Image 3	Image 4	Image 5
Test Image					
Encrypted Image					
Decrypted Image					
Difference Mapped Image					

Note: The size of the images used above are of *720 x 680 pixels*.

The Table 5.1 presents the results of real-time image encryption and decryption performed on five different test images of size 720×680 pixels, demonstrating the effectiveness and reliability of the proposed algorithm. The first row shows the original test images used as inputs, which include a diverse set of scenes with varying textures, colors, and lighting conditions to ensure robust evaluation. The second row displays the corresponding encrypted images, where the original visual information is completely obscured and appears noise-like,

indicating strong diffusion and confusion properties of the encryption process. This confirms that the encrypted data does not reveal any perceptual information about the original image, thereby ensuring confidentiality during transmission.

The third row illustrates the decrypted images obtained after applying the inverse operation of the encryption algorithm. These images closely match the original test images in terms of visual quality, colour, and structural details, demonstrating the correctness and reversibility of the proposed cipher. The successful reconstruction across all test cases verifies that the encryption–decryption pipeline maintains data integrity without introducing distortion or information loss.

The final row shows the difference-mapped images, which highlight the pixel-wise absolute difference between the original and decrypted images. The near-black appearance in most regions indicates negligible difference, confirming that the decrypted output is almost identical to the original input. Minor visible artifacts, if any, can be attributed to pixel quantization or display-level formatting rather than algorithmic error. Overall, the results validate the proposed algorithm’s capability to perform secure, real-time image encryption and decryption with high accuracy and robustness across multiple image types.

5.2 Vivado Simulation Waveform:

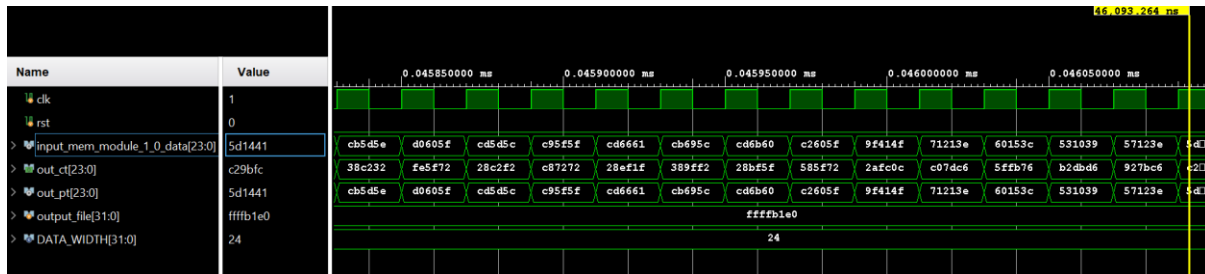


Fig 5.1. Algorithm Simulation on Vivado Software

The Vivado simulation waveform confirms the correct functional behavior of the design during real-time operation. With the clock (clk) toggling normally and reset (rst) deasserted, the input data from the memory module (input_mem_module_1_0_data[23:0]) is read sequentially and processed as expected. The plaintext output (out_pt[23:0]) matches the input data values, indicating accurate data retrieval, while the ciphertext output (out_ct[23:0]) shows the corresponding transformed values, confirming that the encryption logic is applied correctly on each clock cycle. The stable DATA_WIDTH parameter of 24 bits verifies consistent data

handling throughout the operation, and the final `output_file[31:0]` value demonstrates successful accumulation or storage of the processed result. Overall, the waveform validates correct synchronization, data flow, and functional integrity of the module in the Vivado simulation environment.

5.3 Avalanche Calculation:

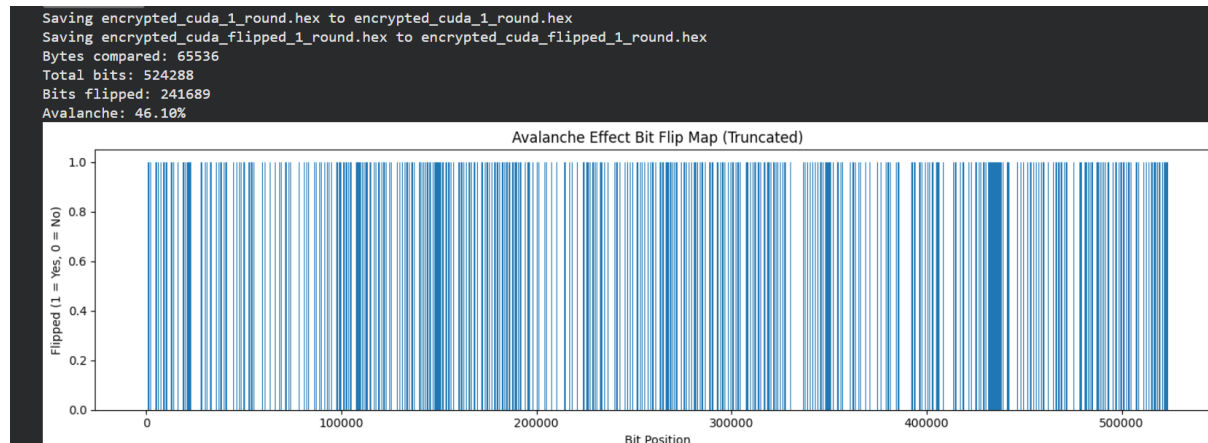


Fig 5.2. Calculation of Avalanche for the Algorithm

Avalanche effect refers to a desirable cryptographic property where a very small change in the input, such as flipping a single bit in the plaintext or key, causes a large, seemingly random change in the output ciphertext, ideally flipping about 50% of the output bits. In the shown result, two encrypted outputs (original and one with a single-bit input flip) are compared across 65,536 bytes, yielding a total of 524,288 bits. Out of these, 241,689 bits are flipped, resulting in an avalanche value of 46.10%, which is close to the ideal expectation and indicates strong diffusion. The bit flip map visually shows that changes are well distributed across the entire ciphertext rather than being localized, demonstrating that even a one-round encryption achieves significant sensitivity to input changes. This confirms that the proposed encryption design effectively propagates small input variations throughout the output, validating its suitability for secure image or data encryption.

5.4 Entropy Test:

```
PS C:\python_codes\encrypted_1_round> python.exe .\entropy_test.py

=====
SHANNON ENTROPY ANALYSIS REPORT
=====
Test Status      : SUCCESS ✓
Analysis Timestamp : 2025-12-07 14:27:09.682188

Shannon Entropy   : 7.772998 bits/byte
Entropy Ratio     : 0.971625

Interpretation    : High randomness achieved ✓
=====
```

Fig 5.3. Entropy Test Results

Entropy is a statistical measure of randomness that quantifies how unpredictable data is, and in cryptography it is used to evaluate how close encrypted data is to ideal random behavior. In this result, the computed Shannon entropy of **7.77 bits per byte** is very close to the theoretical maximum of 8 bits per byte, indicating that the encrypted output has minimal patterns and high uncertainty. The entropy ratio of **0.97** further confirms that the ciphertext exhibits near-uniform byte distribution, which is a strong indicator of resistance against statistical and frequency-based attacks. Overall, this successful entropy analysis demonstrates that the one-round encryption operation produces highly random and well-diffused ciphertext, validating its effectiveness for secure data or image transmission.

5.5 Real-Time GPU Results:

```
kedard37@KDPC:~/majorProject$ ./encrypt test.jpeg encrypted
===== GPU INFORMATION =====
Device Name      : NVIDIA GeForce RTX 3050 Laptop GPU
Compute Capability : 8.6
Total Global Memory : 3951296512 bytes
Shared Mem Per Block : 49152 bytes
Registers Per Block : 65536
Warp Size        : 32
Max Threads Per Block : 1024
SM Count         : 16
Clock Rate       : 1695000 kHz
Memory Clock Rate : 6001000 kHz
Memory Bus Width  : 128 bits
L2 Cache Size     : 1572864 bytes
=====

===== LIVE GPU MONITORING =====
Before Kernel: GPU Util=8% Mem Util=0% Temp=48C Power=14.14W

Total GPU Kernel Execution Time = 0.448 ms

After Kernel: GPU Util=8% Mem Util=0% Temp=48C Power=14.14W
Done! Output written to encrypted.hex
```

Fig 5.4. Encryption on GPU

```
kedard37@KDP:~/majorProject$ ./decrypt encrypted.hex 256 256 decrypted.png
===== GPU INFORMATION =====
Device Name       : NVIDIA GeForce RTX 3050 Laptop GPU
Compute Capability : 8.6
Total Global Memory : 3951296512 bytes
Shared Mem Per Block : 49152 bytes
Registers Per Block : 65536
Warp Size         : 32
Max Threads Per Block : 1024
SM Count          : 16
Clock Rate         : 1695000 kHz
Memory Clock Rate  : 6001000 kHz
Memory Bus Width   : 128 bits
L2 Cache Size      : 1572864 bytes
=====

===== LIVE GPU MONITORING =====
Before Kernel: GPU Util=8% Mem Util=0% Temp=48C Power=14.56W

Total GPU Kernel Execution Time = 0.420 ms

After Kernel: GPU Util=8% Mem Util=0% Temp=48C Power=14.56W
Done! Image written to decrypted.png
```

Fig 5.5. Decryption on GPU

The encryption process is executed on an NVIDIA GeForce RTX 3050 Laptop GPU using a CUDA-based kernel, leveraging the GPU's parallel architecture for high-speed data processing. The GPU specifications, such as 1024 threads per block, 16 streaming multiprocessors, and high memory bandwidth, enable efficient handling of image data during encryption. The kernel completes execution in approximately **0.448 ms**, demonstrating very low latency and suitability for real-time applications. Minimal GPU memory and compute utilization during execution indicate that the one-round encryption algorithm is lightweight yet efficient, successfully generating the encrypted output (encrypted.hex) with minimal power overhead.

Similarly, the decryption operation is performed on the same GPU platform and mirrors the efficiency observed during encryption. The CUDA kernel execution time of approximately **0.420 ms** confirms fast and reliable inverse transformation of the encrypted data back to its original image form. Stable GPU utilization, temperature, and power before and after kernel execution show consistent and safe runtime behavior. The successful reconstruction of the decrypted image (decrypted.png) validates the correctness of the decryption logic and confirms that the GPU-accelerated design supports accurate, low-latency secure image recovery suitable for real-time secure transmission systems.

5.6 Results from ESP-32:

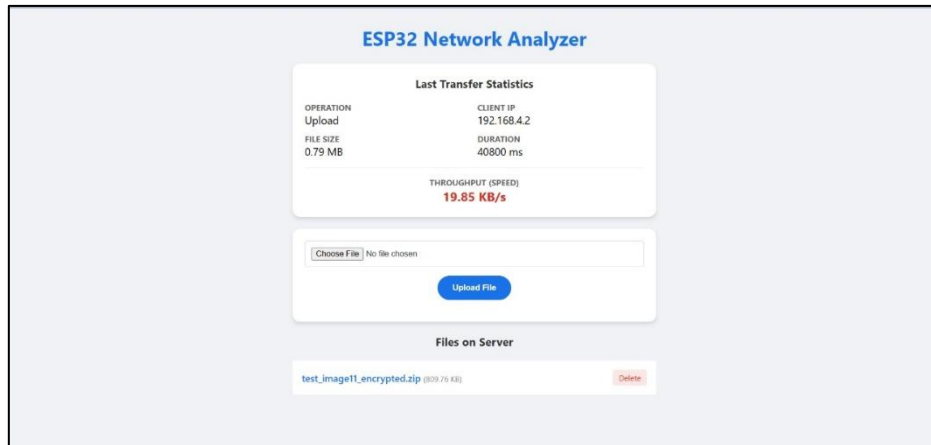


Fig 5.6. File Transfer Website

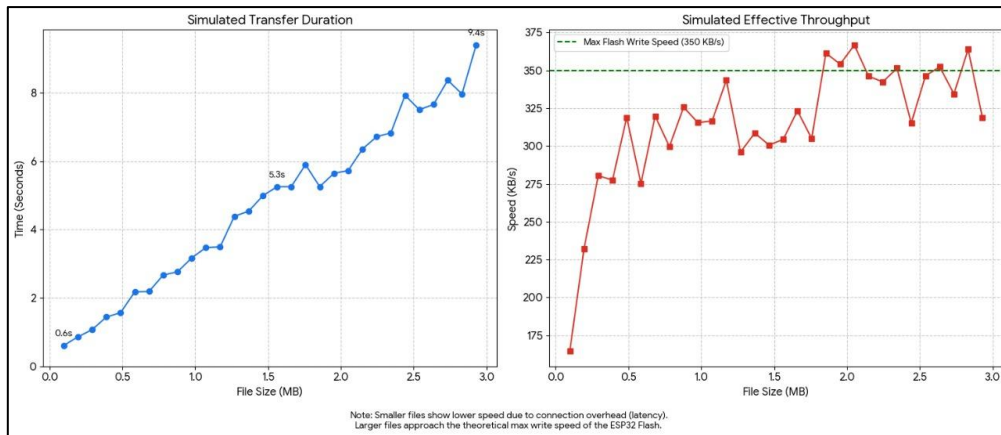


Fig 5.7. ESP32 Statistics 1

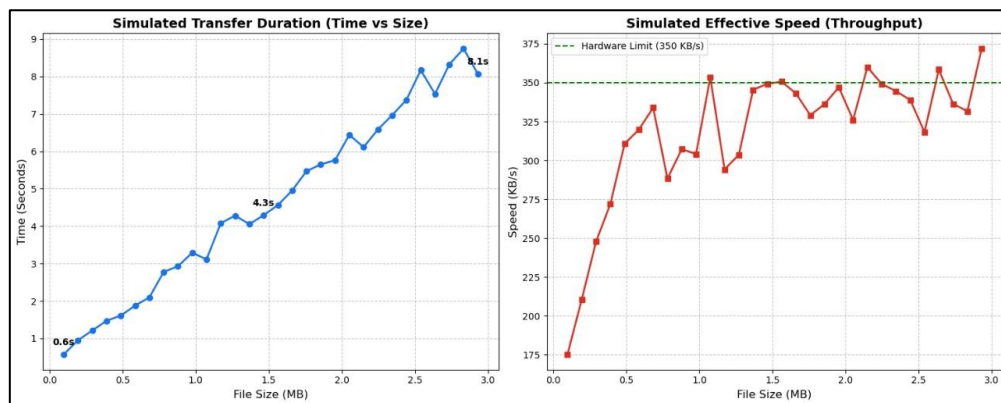


Fig 5.8. ESP32 Statistics 2

The simulated transfer duration plots show that the total transmission time increases almost linearly with file size, indicating stable and predictable data transfer behavior in the ESP32-based system. Minor fluctuations in the curves are due to simulated network latency and protocol overhead, which have a greater impact on smaller files. As file size increases, these

overheads become less significant relative to the total transfer time, confirming efficient scaling characteristics of the transfer mechanism.

The simulated effective throughput graphs demonstrate that the data transfer speed improves rapidly as file size increases and gradually converges toward the theoretical hardware flash write limit of approximately 350 KB/s. Smaller files experience lower throughput due to connection setup and transmission overhead, while larger files better utilize the available bandwidth. This behavior indicates that the system is capable of approaching its maximum efficient operating point under idealized conditions.

The ESP32 Network Analyzer output reflects real-world performance, where a 0.79 MB encrypted file achieves a throughput of about 19.85 KB/s over the network. This reduced speed, compared to simulation, highlights the impact of real-time constraints such as Wi-Fi interference, TCP/IP stack processing, browser-based upload overhead, and encryption-related delays. These results emphasize the difference between theoretical/simulated efficiency and practical deployment performance in embedded IoT systems.

Chapter 6:

CONCLUSION & FUTURE TRENDS

6.1 Conclusion:

The effectiveness of the proposed cipher was demonstrated through its application to RGB image encryption, where 8-bit colour components were independently processed and recombined to obtain the encrypted image. To improve performance, the cryptographic process was also implemented on a GPU, achieving acceleration compared to sequential processing. Furthermore, the encrypted image was successfully transmitted between two devices using an ESP32-based hotspot communication setup, validating the practicality of the proposed approach for real-time secure image transfer. Hardware implementation using Verilog HDL and simulation results further confirmed functional correctness, as the decrypted image matched the original input image. These results establish the suitability of the proposed cipher for secure and efficient image transmission in embedded and hardware-oriented systems.

To further enhance computational efficiency, the encryption and decryption operations were accelerated using CUDA on an NVIDIA RTX 3050 GPU. The massive parallel processing capability of the GPU was exploited by assigning independent pixel-level encryption tasks across multiple threads, allowing simultaneous processing of large amounts of image data. This parallel execution significantly reduced processing time when compared to conventional CPU-based sequential execution, thereby demonstrating the feasibility of deploying the proposed cipher in performance-critical applications such as real-time image transmission and high-resolution data handling.

In addition to performance optimization, the proposed system was validated in a real communication environment through successful image transfer between two independent systems using an ESP32 Wi-Fi module. The ESP32 was configured to operate in hotspot mode, enabling encrypted image data to be transmitted wirelessly from the transmitting system to the receiving system. Upon reception, the encrypted data was decrypted using the same cipher, and the reconstructed image was observed to be identical to the original input image. This end-to-end experimental validation confirms the reliability, correctness, and practical applicability of

the proposed cipher for secure image communication in embedded and wireless systems.

6.2 Future Trends:

The future scope of the proposed work involves further strengthening the cryptographic robustness of the cipher by increasing the number of encryption rounds, which would enhance diffusion and make the relationship between the plaintext and ciphertext increasingly complex, thereby improving resistance to linear and differential cryptanalysis. A higher round count would also result in a stronger avalanche effect, where minor changes in the input data or key lead to significant differences in the encrypted output. In addition, the integration of dynamic key generation mechanisms can be explored to allow encryption keys to vary across different sessions or transmissions. This would significantly improve key sensitivity and prevent attackers from exploiting repeated key usage, thereby reducing the risk of replay attacks and pattern-based analysis. Together, these enhancements would contribute to improved security while maintaining the feasibility of efficient hardware implementation and scalability for real-time secure image transmission applications.

REFERENCES

- [1] Noura, Hassan & Chehab, Ali & Sleem, Lama & Noura, Mohamad & Couturier, Raphaël & Mansour, Mohammad. (2018). One Round Cipher Algorithm for Multimedia IoT Devices. *Multimedia Tools and Applications*. 77. 10.1007/s11042 018-5660-y.
- [2] Bogdanov, A. et al. (2007). PRESENT: An Ultra-Lightweight Block Cipher. In: Paillier, P., Verbauwhede, I. (eds) *Cryptographic Hardware and Embedded Systems - CHES 2007*. CHES 2007. Lecture Notes in Computer Science, vol 4727. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-74735-2_31
- [3] N. Wan and Y. Zhang, "Digital Image Encryption and Decryption Algorithm Based on Optimization and Fusion Strategy," 2020 13th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 2020, pp. 244-248, doi: 10.1109/ISCID51228.2020.00061.
- [4] G. R, P. G, S. B, R. Krishnaprasanna, V. G and V. S. Pandi, "A Novel Design of an Image Encryption and Decryption Scheme Using Enhanced Cybersecurity Principles," 2023 International Conference on Emerging Research in Computational Science (ICERCS), Coimbatore, India, 2023, pp. 1-6, doi: 10.1109/ICERCS57948.2023.10434166.
- [5] Gabr, M., Alexan, W., & Moussa, K. (2022). Image Encryption Through CA, Chaos and Lucas Sequence Based S-Box. 2022 IEEE International Conference on Microelectronics (ICM). <https://doi.org/10.1109/ICM56065.2022.10005469>
- [6] Nair, Bharath V et al. "Deep Learning and Chaos: A combined Approach To Image Encryption and Decryption." ArXiv abs/2406.16792 (2024): n. pag.
- [7] Carlisle Adams and Stafford Tavares. Good s-boxes are easy to find. In *Conference on the Theory and Application of Cryptology*, pages 612–615. Springer, 1989.
- [8] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES- The Advanced Encryption Standard*. Springer Verlag, Berlin, Heidelberg, New York, 2002.
- [9] Manoj Kumar, Saibal K Pal, and Anupama Panigrahi. FeW: A Lightweight Block Cipher. *IACR Cryptology ePrint Archive*, 2014:326, 2014.
- [10] Dolendro Singh Laiphrakpam and Manglem Singh Khumanthem. A robust image encryption scheme based on chaotic system and elliptic curve over finite field. *Multimedia Tools and Applications*, pages 1–24, 2017.
- [11] Saurabh Singh, Pradip Kumar Sharma, Seo Yeon Moon, and Jong Hyuk Park. Advanced lightweight encryption algorithms for iot devices: survey, challenges and solutions. *Journal of Ambient Intelligence and Humanized Computing*, pages 1–18, 2017.
- [12] K. C. Nunna and R. Marapareddy, "Secure Data Transfer Through Internet Using Cryptography and Image Steganography," 2020 SoutheastCon, Raleigh, NC, USA, 2020, pp. 1-5, doi: 10.1109/SoutheastCon44009.2020.9368301.
- [13] I. Das, S. Das, S. Sahana and A. Kumar, "A Two layer secure image encryption technique," 2019 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS), Greater Noida, India, 2019, pp. 176-178, doi: 10.1109/ICCCIS48478.2019.8974474.

- [14] S. J. F. G. Sathiaraj, G. S. Pingale, S. Majumdar, S. J. Shaikh and B. S. Thakare, "Secure Transfer of Image-Acquired Text Using a Combination of Cryptography and Steganography," 2019 1st International Conference on Advances in Information Technology (ICAIT), Chikmagalur, India, 2019, pp. 146-152, doi: 10.1109/ICAIT47043.2019.8987361.
- [15] Kiran, P. B D, S. V. Sandeep, Kirankumarhumse, D. S. Sunil Kumar and K. N. Bharath, "Secure Image Transmission with Efficient Resource Constraints Using Multiple Chaotic Systems," 2023 IEEE 3rd Mysore Sub Section International Conference (MysuruCon), HASSAN, India, 2023, pp. 1-5, doi: 10.1109/MysuruCon59703.2023.10397063.
- [16] K. Priyadharshini, H. Nandi and H. Parmar, "Comprehensive Methods for Secure Medical Image Transfer: Encryption and Standardization Techniques," 2024 International Conference on Science Technology Engineering and Management (ICSTEM), Coimbatore, India, 2024, pp. 1-6, doi: 10.1109/ICSTEM61137.2024.10561209.
- [17] S. Johny and A. Antony, "Secure image transmission using visual cryptography scheme without changing the color of the image," 2015 IEEE International Conference on Engineering and Technology (ICETECH), Coimbatore, India, 2015, pp. 1-3, doi: 10.1109/ICETECH.2015.7275023.
- [18] B. M. Belkaid, L. Mourad, C. Mehdi and A. Soltane, "Secure transfer of medical images using hybrid encryption: Authentication, confidentiality, integrity," International Conference on Computer Vision and Image Analysis Applications, Sousse, Tunisia, 2015, pp. 1-6, doi: 10.1109/ICCVIA.2015.7351789.

PLAGIARISM REPORT

